

Optimization part

3-to_8 decoder

Initially, we considered implementing the 3-to-8 decoder using multiple 2-to-4 decoders combined with additional logic through processes. However, this added unnecessary resource usage. Instead, we opted for a direct 3-to-8 decoder implementation using a simple case statement structure. This approach not only simplified the design but also minimized logic depth and improved synthesis results by reducing the use of LUTs and multiplexers.

slw_clk

The main difference between the two slow clock designs is in how they generate the output signal. The earlier counter-based design uses a comparison method, where the counter counts up to a fixed value and then toggles the output before resetting. This approach provides precise control over the output frequency, making it suitable when an exact timing signal (like 1 Hz) is required, but it uses more logic resources. In contrast, the newer design uses the most significant bit (MSB) of a continuously running binary counter as the output. This method is much simpler and more efficient in terms of hardware usage, as it avoids comparison logic, but it produces only an approximate frequency rather than an exact one.

Mux_8_4bit

The design using the `with-select` statement is more optimized for FPGA implementation because it allows the synthesis tool to directly map the multiplexer into LUTs, resulting in lower resource usage, simpler hardware, and faster performance. In contrast, the normal design uses a decoder and multiple tri-state buffers, which increases hardware complexity and resource usage. Additionally, modern FPGAs do not efficiently support internal tri-state logic, so this design is usually converted into regular logic anyway, making it less efficient. Overall, the first design is optimized in terms of **area, speed, and simplicity**, while the second is more structural but less resource-efficient.

Normal one

```
-- Company:
-- Engineer:
--
-- Create Date: 05/13/2025 02:37:41 PM
-- Design Name:
-- Module Name: Mux_2_4bit - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
```

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using

-- arithmetic functions with Signed or Unsigned values

--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating

-- any Xilinx leaf cells in this code.

--library UNISIM;

--use UNISIM.VComponents.all;

entity Mux_8_4bit_new is

Port (A0 : in STD_LOGIC_VECTOR (3 downto 0);

A1 : in STD_LOGIC_VECTOR (3 downto 0);

A2 : in STD_LOGIC_VECTOR (3 downto 0);

A3 : in STD_LOGIC_VECTOR (3 downto 0);

A4 : in STD_LOGIC_VECTOR (3 downto 0);

A5 : in STD_LOGIC_VECTOR (3 downto 0);

A6 : in STD_LOGIC_VECTOR (3 downto 0);

A7 : in STD_LOGIC_VECTOR (3 downto 0);

Sel : in STD_LOGIC_VECTOR (2 downto 0);

Y : out STD_LOGIC_VECTOR (3 downto 0));

end Mux_8_4bit_new;

architecture Behavioral of Mux_8_4bit_new is

component Tri_State_buffer_4bit is

port (I : in STD_LOGIC_VECTOR (3 downto 0);

EN : in STD_LOGIC;

M : out STD_LOGIC_VECTOR (3 downto 0));

end component;

component Decoder_3_to_8 is

port(I : in STD_LOGIC_VECTOR (2 downto 0);

EN : in STD_LOGIC;

Y : out STD_LOGIC_VECTOR (7 downto 0));

end component;

```
signal Enable : STD_LOGIC_VECTOR(7 downto 0);
```

```
begin
```

```
Decoder_3_to_8_0: Decoder_3_to_8
```

```
port map( I => Sel,  
          EN => '1',  
          Y => Enable  
        );
```

```
buffer_0: Tri_State_buffer_4bit
```

```
port map( I => A0,  
          EN => Enable(0),  
          M => Y  
        );
```

```
buffer_1: Tri_State_buffer_4bit
```

```
port map( I => A1,  
          EN => Enable(1),  
          M => Y );
```

```
buffer_2: Tri_State_buffer_4bit
```

```
port map( I => A2,  
          EN => Enable(2),  
          M => Y );
```

```
buffer_3: Tri_State_buffer_4bit
```

```
port map( I => A3,  
          EN => Enable(3),  
          M => Y );
```

```
buffer_4: Tri_State_buffer_4bit
```

```
port map( I => A4,  
          EN => Enable(4),  
          M => Y );
```

```
buffer_5: Tri_State_buffer_4bit
```

```
port map( I => A5,  
          EN => Enable(5),  
          M => Y );
```

```
buffer_6: Tri_State_buffer_4bit
```

```
port map( I => A6,  
          EN => Enable(6),  
          M => Y );
```

```
buffer_7: Tri_State_buffer_4bit
```

```
port map( I => A7,  
          EN => Enable(7),
```

```
M => Y );
```

```
end Behavioral;
```

Optimied one.....

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Mux_8_4bit is
```

```
Port ( A0, A1, A2, A3, A4, A5, A6, A7 : in STD_LOGIC_VECTOR (3 downto 0);
```

```
      Sel : in STD_LOGIC_VECTOR (2 downto 0);
```

```
      Y : out STD_LOGIC_VECTOR (3 downto 0));
```

```
end Mux_8_4bit;
```

```
architecture Behavioral of Mux_8_4bit is
```

```
begin
```

```
-- Using a 'with-select' statement for optimal LUT mapping
```

```
with Sel select
```

```
Y <= A0 when "000",
```

```
    A1 when "001",
```

```
    A2 when "010",
```

```
    A3 when "011",
```

```
    A4 when "100",
```

```
    A5 when "101",
```

```
    A6 when "110",
```

```
    A7 when "111",
```

```
    "0000" when others;
```

```
end Behavioral;
```

Add_Sub_unit-----

Zero Flag: In your processor, instructions like **JZR** (Jump if Zero) depend solely on the value in the register being zero. The Carry bit shouldn't block that jump.

Logic Resource: By using **STD_LOGIC_VECTOR** for **z**, the code is cleaner and easier for Vivado/Quartus to map to **LUTs** during synthesis.

Sign Detection: The corrected overflow logic ensures that if you add \$7 + 1\$ (both positive in 4-bit), you correctly get an overflow flag because the result is \$-8\$ (negative in 4-bit).

Full adder -----

Optimized code

architecture Behavioral of FA is

begin

-- Direct logic implementation

-- Sum logic: XORing all three inputs

S <= A XOR B XOR C_in;

-- Carry out logic: Majority function

-- Optimized to calculate in parallel with the Sum

C_out <= (A AND B) OR (C_in AND (A XOR B));

end Behavioral;

If we build a Full Adder without nesting Half Adders is generally better for FPGA optimization because it reduces gate delay and maps more cleanly to a single **LUT (Look-Up Table)**.